

# QuickSelectAlgorithmAndCRUDOperationsOnTuple

24a85a0506 · 14 de junio de 2026



In [2]:

```
def quick_select(arr, left, right):
    pivot = arr[right]
    ploc = left
    for j in range(left, right):
        if arr[j] < pivot:
            arr[ploc], arr[j] = arr[j], arr[ploc]
            ploc += 1

    arr[ploc], arr[right] = arr[right], arr[ploc]
    return ploc

def kth_smallest(arr, left, right, k):
    while left <= right:
        p = quick_select(arr, left, right)

        if p == k:
            return arr[p]
        elif p < k:
            left = p + 1
        else:
            right = p - 1

    return None

arr = [10, 20, 30, 40, 50, 60, 70]
k = 3

ans = kth_smallest(arr, 0, len(arr) - 1, k - 1)
print(ans)
```

Out [2]:

30

In [ ]:

```
def quick_select(arr, left, right):
    pivot = arr[right]
    ploc = left
    for j in range(left, right):
        if arr[j] < pivot:
            arr[ploc], arr[j] = arr[j], arr[ploc]
            ploc += 1

    arr[ploc], arr[right] = arr[right], arr[ploc]
    return ploc

def kth_largest(arr, left, right, k):
    while left <= right:
        p = quick_select(arr, left, right)

        if p == n-k:
            return arr[p]
        elif p < n-k:
            left = p + 1
        else:
            right = p - 1

    return None

arr = [10, 20, 30, 40, 50, 60, 70]
k = 3
```

```
ans = kth_largest(arr, 0, len(arr) - 1, k)
print(ans)
```

```
In [ ]:
stack = []

def push(x):
    stack.append(x)
    return stack[::-1]

def pop():
    if len(stack)==0: return None
    else :
        x = stack[-1]
        stack.pop()
        traverse()
        return x

def size(): return len(stack)

def isEmpty():
    if len(stack)==0: return "Stack is Empty"
    else: return f"Stack is not empty with size{len(stack)}"

def top():
    if len(stack)==0: return "No top"
    else: return stack[0]
def traverse():
    return stack[::-1]

while(True):
    print("-----\n1.push\n2.pop\n3.size\n4.isEmpty\n5.top\n6.exit\n-----")

    case = int(input("Enter Number To perform Opeartion:"))

    match case :
        case 1:
            x = int(input("Enter Element TO Insert Stack "))
            print("After Pushing Element Stack will be",push(x))

        case 2:
            print(pop())
        case 3:
            print(size())
        case 4:
            print(isEmpty())
        case 5:
            print(top())
        case 6:
            break
        case _:
            print(traverse())
```

```
In [5]:
l = ["a","b","c"]
l1 = [1, 2, 3]
l2 = list(zip(l,l1))
```

```
In [6]:
print(l2)
```

```
Out [6]:
[('a', 1), ('b', 2), ('c', 3)]
```

In [7]:

```
l2[0]
```

Out [7]:

```
('a', 1)
```

In [8]:

```
type(l2[0])
```

Out [8]:

```
tuple
```

In [11]:

```
l3 = list(zip(l1 , l2))  
l3
```

Out [11]:

```
[(1, ('a', 1)), (2, ('b', 2)), (3, ('c', 3))]
```

In [12]:

```
n = tuple(list(map(int , input().split())))  
print(n)  
print(type(n))
```

Out [12]:

```
(1, 2, 3, 4, 5)  
<class 'tuple'>
```

In [13]:

```
# update  
n = list(n)  
print(type(n))  
  
n[0] = 80  
  
n = tuple(n)  
print(type(n) , n )
```

Out [13]:

```
<class 'list'>  
<class 'tuple'> (80, 2, 3, 4, 5)
```